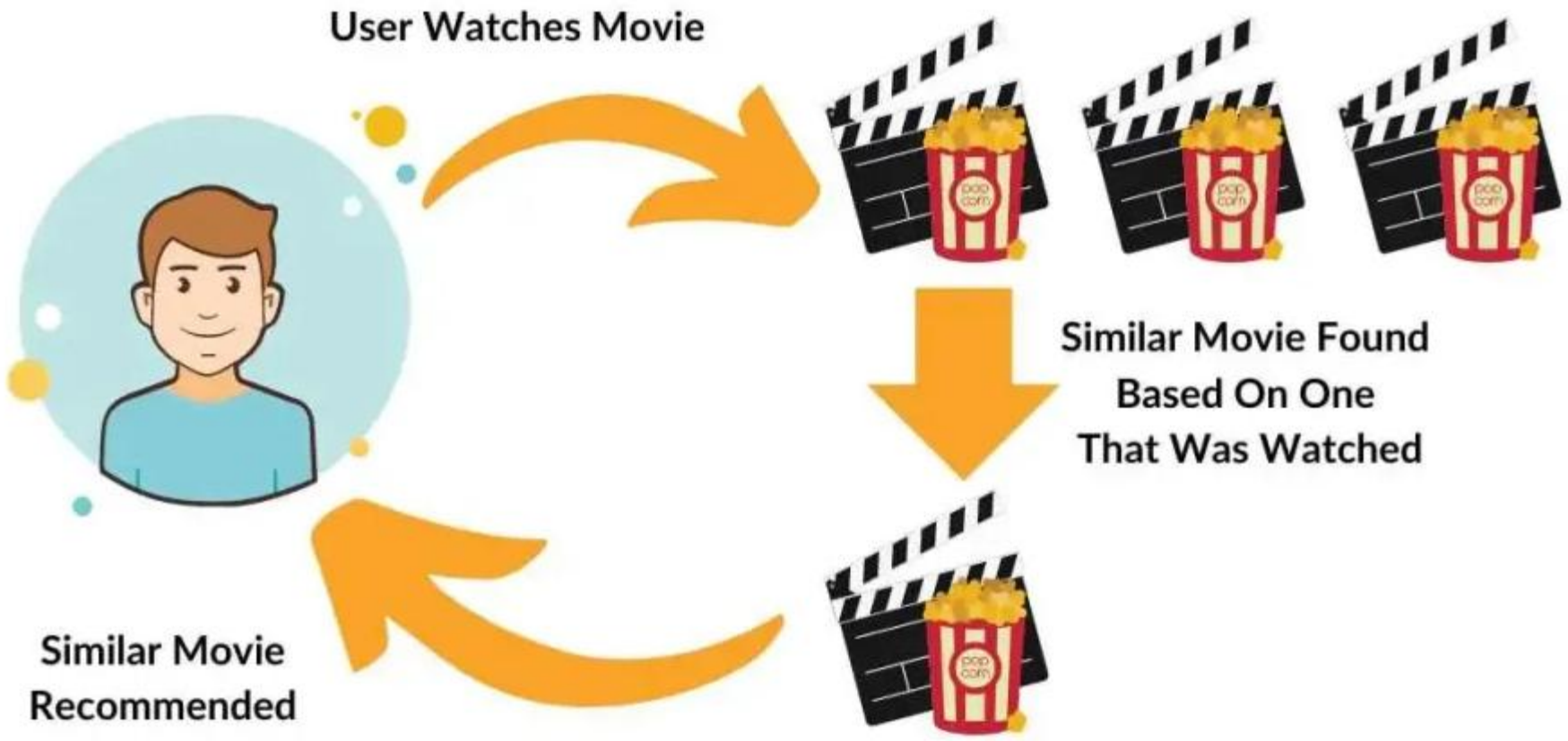
Term Project: Backend Application with PostgreSQL

Building a content recommendation application



Team6

2024-23352 한정관
2024-28634 김주영
2024-25545 이서혁
2022-26826 박승과

1. Rationale for the project – OTT가 미디어 시장을 점유 할 수 있었던 이유

1차(2011)
종편 개국·케이블채널 세력 확장

- 나영석 PD('1박2일'·KBS) → CJ ENM
- 신원호 PD('남자의 자격') → CJ ENM
- 여운혁PD('무릎팍도사'·MBC) → JTBC

2차(2015)
한류 '중국몽'

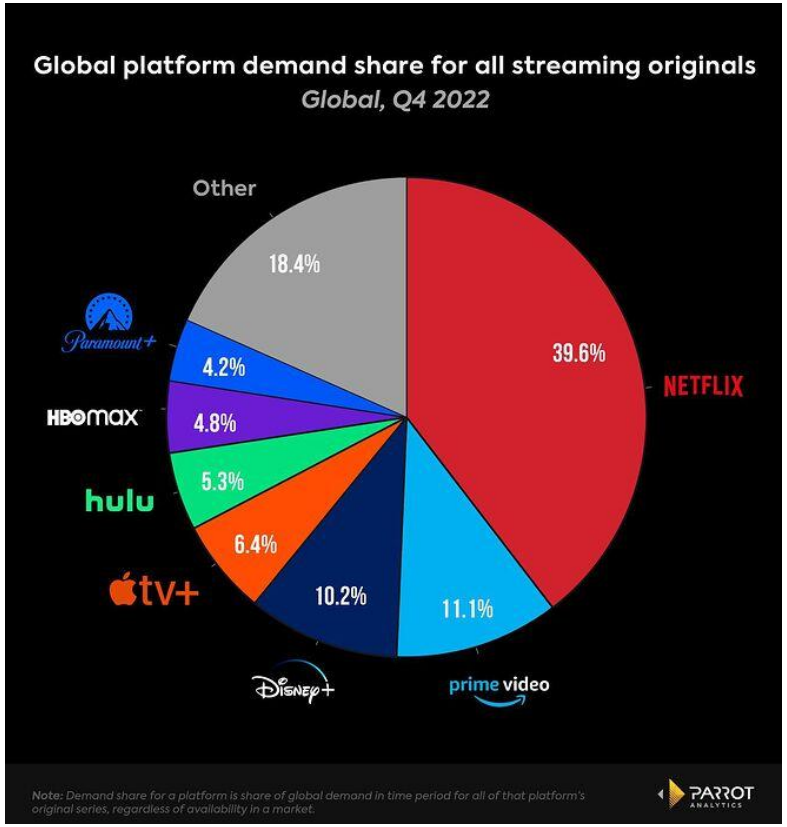
- 김영희PD('위대한 탄생'·MBC)
- 이민호PD('나는 가수다'·MBC)

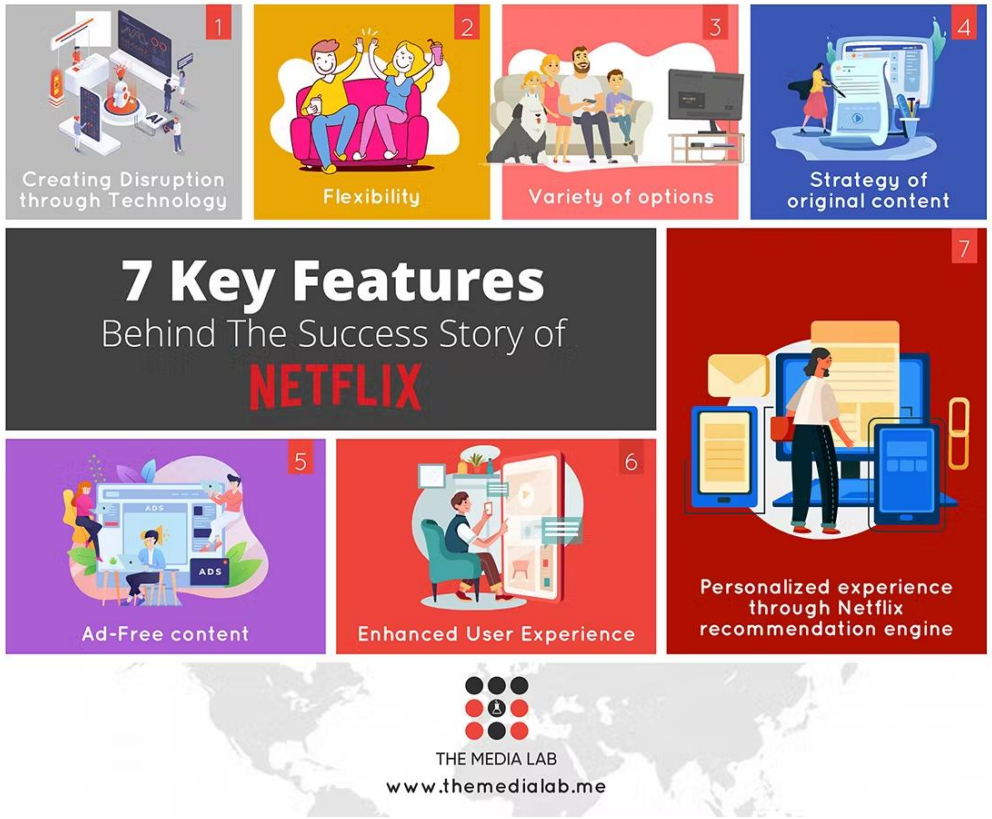
→ 중국서 독립제작

3차(2021~)
OTT 향해 독립

- 김태호 PD('무한도전'·MBC)
- 주동민 PD('펜트하우스'·SBS)
- 차영훈 PD('동백꽃 필 무렵'·KBS)

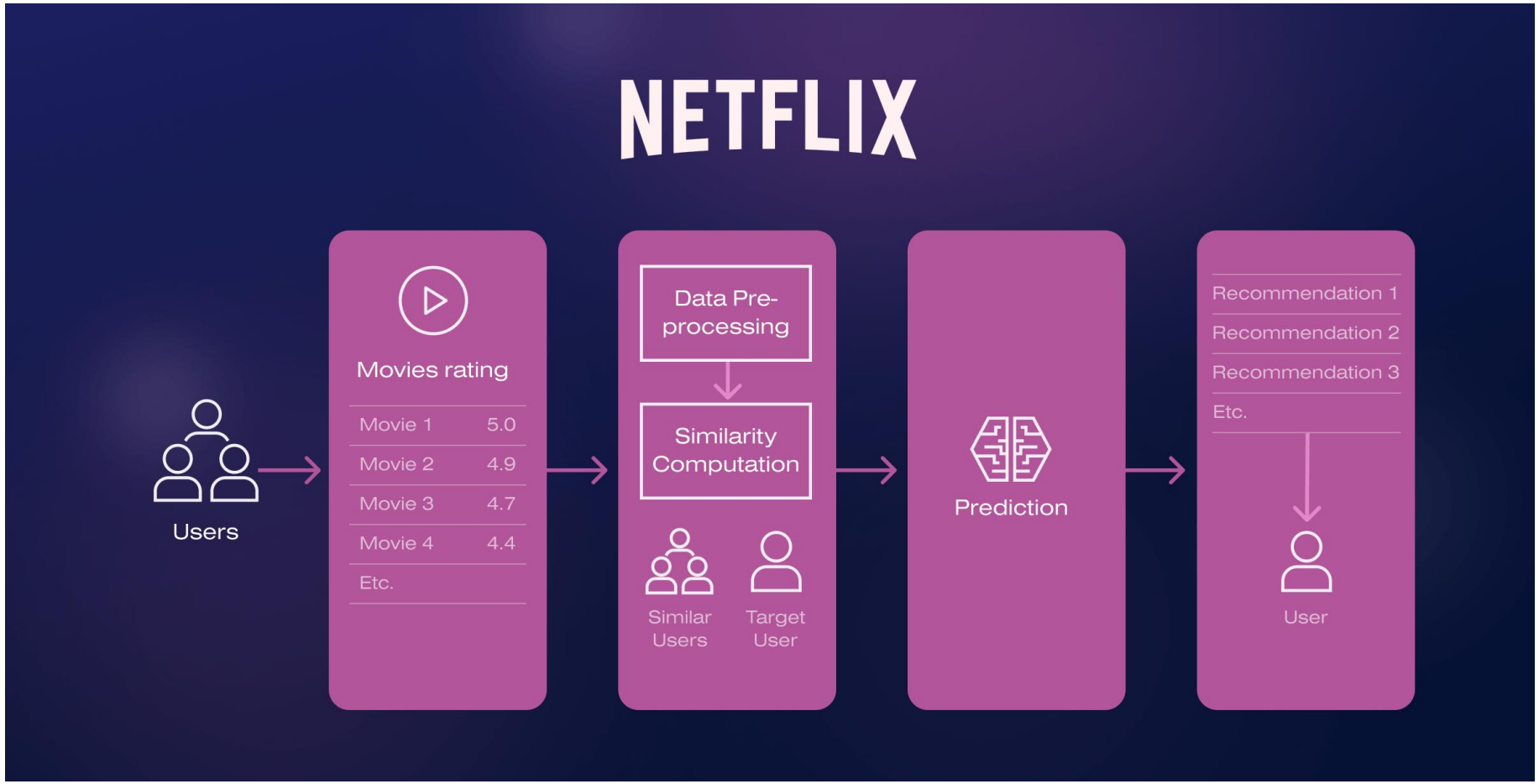
→ 프리랜서





https://n.news.naver.com/mnews/article/469/0000628886
https://m.kinolights.com/community/post/3kDFtIYBD581XP2uzwTG#poster
https://www.themedialab.me/7-key-factors-behind-success-story-netflix/

1. Rationale for the project – OTT가 미디어 시장을 점유 할 수 있었던 이유



Rank by rating / content-based recommendation / collaborative filtering-based recommendation

https://soloway.tech/blog/implementing-machine-learning-in-e-commerce-personalization-and-recommendation-systems/

2. Object of the project

최종목표	Content recommendation application 구축
세부목표	User / Movie 정보 Insert / Delete / Update 구축 Rating system 구축 Rank view 구현 Cold-Start Users 도출 Content-based recommendation 구현 Collaborative filtering-based recommendation 구현

2024-23352 한정관	2024-28634 김주영	2024-25545 이서혁	2022-26826 박승과
Input data 생성	프로그램 구현	프로그램 구현	보고서 작성 및 발표

3. Database schema

movies 30 rows

mid	title	director	nationality	release_year	profit	summary
1	John Wick: Chapter 4	Chad Stahelski	USA	2023	328,000,000	John Wick faces new enemies in his fight for freedom. High-octane action sequences ensue.
2	No Time to Die	Cary Joji Fukunaga	USA	2021	774,000,000	James Bond comes out of retirement to face a new villain. Thrilling espionage action unfolds.
3	Black Widow	Cate Shortland	USA	2021	379,000,000	Natasha Romanoff confronts her past as a spy. Explosive stunts and fight scenes dominate.
4	The Suicide Squad	James Gunn	USA	2021	167,000,000	A new team of anti-heroes embark on a dangerous mission. Dark humor and action abound.
5	F9	Justin Lin	USA	2021	726,000,000	Dom Toretto and his crew face new threats from the past. High-speed car chases and action scenes.
6	Shang-Chi and the Legend of the Ten Rings	Destin Daniel Cretton	USA	2021	432,000,000	Shang-Chi discovers his heritage and faces the Ten Rings. Martial arts and action-packed sequences.
7	On Your Wedding Day	Lee Seok-geun	South Korea	2018	11,000,000	A man receives a wedding invitation from his first love, sparking memories of their high school romance and his unresolved feelings.
⋮	⋮	⋮	⋮	⋮	⋮	⋮

users
15 rows

uid	uname	gender	age
1	Sofia	F	22
2	Liam	M	45
3	Eugene	F	19
4	Hiroshi	M	34
5	Maria	F	27
6	Ahmed	M	28
7	Yana	F	21
⋮	⋮	⋮	⋮

ratings
230 rows

uid	mid	rating	time
1	1	4.5	2024-06-03
1	2	4.0	2024-06-04
1	4	4.0	2024-05-31
1	5	4.5	2024-04-30
1	6	4.0	2024-05-29
1	7	2.0	2024-05-28
1	8	3.5	2024-04-27
⋮	⋮	⋮	⋮

4. Conventional Scenarios – 기본적인 기능

```
Possible commands:
-----------------------------------------------
★ Fetch [ user / movie / rating ]
★ Show [ users / movies / ratings ]
★ Display [ user history / movie ratings ]
-----------------------------------------------
★ Insert [ user / movie / rating ]
★ Update [ user / movie / rating ]
★ Delete [ user / movie / rating ]
-----------------------------------------------
★ Rank [ weekly / monthly ]
-----------------------------------------------
★ Get [ cu / cold-start users ]
★ Rec [ cb / content-based ]
★ Rec [ cf / collaborative filtering ]
-----------------------------------------------
★ Quit

>>>
```

해당 혹은 전체 User / movie / Rating의 정보를 요청 경우

새로운 User / movie / Rating을 추가하는 경우

User / movie / Rating을 갱신 혹은 삭제하는 경우

해당 User / movie의 이력을 요청하는 경우

4. Conventional Scenarios – 기본적인 기능

```
>>> Insert user

name: SG park
gender [F/M/U]: M
age: 31

Inserted user: (16, 'SG park', 'M', 31)
>>> Fetch user

user id: 16

Fetched user: (16, 'SG park', 'M', 31)
```

```
>>> Insert movie

title: Gone with the Wind
director: Victor Fleming
nationality: United States
release_year: 1939
profit: 390000000
summary: On September 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney,
and film editor Hal Kern drove out to Riverside, California to preview the film at the F
ox Theatre.

Inserted movie: (31, 'Gone with the Wind', 'Victor Fleming', 'United States', 1939, 3900
00000, 'On September 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney, a
nd film editor Hal Kern drove out to Riverside, California to preview the film at the Fo
x Theatre.')
>>> Fetch movie

movie id: 31

Fetched movie: (31, 'Gone with the Wind', 'Victor Fleming', 'United States', 1939, 39000
0000, 'On September 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney, an
d film editor Hal Kern drove out to Riverside, California to preview the film at the Fox
 Theatre.')
```

```
>>> Insert rating

user id: 16
movie id: 31
rating: 5
time: 2024-06-12

Inserted rating: (16, 31, 5.0, datetime.date(2024, 6, 12))
movie id: 31

Movie information: (31, 'Gone with the Wind', 'Victor Fleming', 'United States', 1939, 3
90000000, 'On September 9, 1939, Selznick, his wife, Irene, investor John "Jock" Whitney
, and film editor Hal Kern drove out to Riverside, California to preview the film at the
 Fox Theatre.')

Activity history:
(16, 'SG park', 5.0, datetime.date(2024, 6, 12))
```

해당 혹은 전체 User / movie / Rating의 정보를 요청 경우
새로운 User / movie / Rating을 추가하는 경우

4. Conventional Scenarios – main.py

```python
elif (query == "Insert user"):
    uname = input("name: ")
    gender = input("gender [F/M/U]: ")
    age = input("age: ")
    try:
        ret = op.insert_user(uname, gender, age)
        print(f"\nInserted user: {ret}\n")
    except Exception:
        print("\nWrong values!\n")

elif (query == "Insert movie"):
    title = input("title: ")
    director = input("director: ")
    nationality = input("nationality: ")
    release_year = input("release_year: ")
    profit = input("profit: ")
    summary = input("summary: ")
    try:
        ret = op.insert_movie(title, director, nationality, release_year, profit, summary)
        print(f"\nInserted movie: {ret}\n")
    except Exception:
        print("\nWrong values!\n")

elif (query == "Insert rating"):
    uid = input("user id: ")
    try: int(uid)
    except ValueError:
        print("\nWrong value!\n")
        continue
    mid = input("movie id: ")
    try: int(mid)
    except ValueError:
        print("\nWrong value!\n")
        continue
    rating = input("rating: ")
    time = input("time: ")
    try:
        ret = op.insert_rating(uid, mid, rating, time)
        print(f"\nInserted rating: {ret}\n")
    except Exception:
        print("\nWrong values!\n")
```

```python
# Keyboard Input이 Insert user인 경우
        # User 이름 입력 요청
        # User 성별 입력 요청
        # User 나이 입력 요청
        # 잘못된 값 입력 방지를 위한 예외처리
                # operations.py의 insert_user 함수 호출하여 User 정보 추가
                # 추가된 User 정보 출력

                # 예외 발생시에 오류 메시지 출력

# Keyboard Input이 Insert movie인 경우
        # movie 제목 입력 요청
        # movie 감독 입력 요청
        # movie 국가 입력 요청
        # movie 연도 입력 요청
        # movie 수익 입력 요청
        # movie 요약 입력 요청
        # 잘못된 값 입력 방지를 위한 예외처리
                # operations.py의 insert_movie 함수 호출하여 Movie 정보 추가
                # 추가된 Movie 정보 출력

                # 예외 발생시에 오류 메시지 출력

# Keyboard Input이 Insert rating인 경우
        # rating할 User ID 입력 요청
        # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 uid를 정수로 변환 시도
        # 정수 변환 실패 시 예외 발생
                # 사용자에게 오류 메시지 출력
                # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
        # rating 매겨질 Movie ID 입력 요청
        # 사용자가 정수가 아닌 값을 입력할 경우에는 입력된 mid를 정수로 변환 시도
        # 정수 변환 실패 시 예외 발생
                # 사용자에게 오류 메시지 출력
                # 다음 명령어 입력 대기 (현재 명령어 처리 종료)
        # Movie의 rating 입력 요청
        # rating 날짜 입력 요청
        # 잘못된 값 입력 방지를 위한 예외처리
                # operations.py의 insert_rating 함수 호출하여 Rating 정보 추가
                # 추가된 Rating 정보 출력

                # 예외 발생시에 오류 메시지 출력
```

4. Conventional Scenarios – operations.py

```python
def insert_user(uname, gender, age):                                    # Keyboard Input을 통해 받은 User 정보를 이용하여 새로운 User를 추가하는 함수
    conn = connect_to_db()                                                  # PostgreSQL에 연결
    cur = conn.cursor()                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    query = """
        INSERT INTO users (uname, gender, age)
        VALUES (%s, %s, %s)
        RETURNING *
    """                                                                     # 입력 값을 이용하여 새로운 User를 추가 가능한 SQL 쿼리문 생성
    cur.execute(query, (uname, gender, age))                                # SQL 쿼리 실행
    ret = cur.fetchone()                                                    # 추가한 행 가져오기
    disconnect(cur, conn)                                                   # PostgreSQL에 연결 종료
    return ret                                                              # 추가된 User 정보 리턴

def insert_movie(title, director, nationality, release_year, profit, summary):   # Keyboard Input을 통해 받은 Movie 정보를 이용하여 새로운 Movie를 추가하는 함수
    conn = connect_to_db()                                                  # PostgreSQL에 연결
    cur = conn.cursor()                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    embedding = embeddings_model.embed_query(summary)                       # OpenAI 임베딩 모델을 사용하여 Movie 요약 정보에 대한 임베딩 벡터 생성

    query = """
        INSERT INTO movies (title, director, nationality, release_year, profit, summary, embedding)
        VALUES (%s, %s, %s, %s, %s, %s, %s)
        RETURNING *
    """                                                                     # 입력 값을 이용하여 새로운 Movie를 추가 가능한 SQL 쿼리문 생성
    cur.execute(query, (title, director, nationality, release_year, profit, summary, embedding))   # SQL 쿼리 실행
    ret = cur.fetchone()[:-1]                                               # 추가한 행 가져오기
    disconnect(cur, conn)                                                   # PostgreSQL에 연결 종료
    return ret                                                              # 추가된 Movie 정보 리턴

def insert_rating(uid, mid, rating, time):                              # Keyboard Input을 통해 받은 Rating 정보를 이용하여 새로운 Rating를 추가하는 함수
    conn = connect_to_db()                                                  # PostgreSQL에 연결
    cur = conn.cursor()                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    query = """
        INSERT INTO ratings (uid, mid, rating, time)
        VALUES (%s, %s, %s, %s)
        RETURNING *
    """                                                                     # 입력 값을 이용하여 새로운 Rating를 추가 가능한 SQL 쿼리문 생성
    cur.execute(query, (uid, mid, rating, time))                            # SQL 쿼리 실행
    ret = cur.fetchone()                                                    # 추가한 행 가져오기
    disconnect(cur, conn)                                                   # PostgreSQL에 연결 종료
    return ret                                                              # 추가된 Rating 정보 리턴
```

5. Main Scenario 1

Movie의 Rating에 기반한 Weekly 혹은 Monthly Ranking의 정보를 요청하는 경우

```
>>> Rank weekly

(5, 'F9', Decimal('5.00'))
(17, 'The Wailing', Decimal('5.00'))
(19, 'Encanto', Decimal('4.83'))
(27, 'Flee', Decimal('4.75'))
(29, 'The Sparks Brothers', Decimal('4.75'))
(21, 'Zootopia', Decimal('4.75'))
(13, 'A Quiet Place Part II', Decimal('4.75'))
(15, 'Malignant', Decimal('4.75'))
(25, 'The Rescue', Decimal('4.67'))
(8, 'Decision to Leave', Decimal('4.50'))
(26, 'Summer of Soul', Decimal('4.50'))
(3, 'Black Widow', Decimal('4.50'))
(23, 'Madagascar', Decimal('4.50'))
(28, 'Val', Decimal('4.25'))
(16, 'It', Decimal('4.25'))
(2, 'No Time to Die', Decimal('4.25'))
(9, 'Begin Again', Decimal('4.25'))
(14, 'The Conjuring: The Devil Made Me Do It', Decimal('4.25'))
(20, 'Luca', Decimal('4.25'))
(10, 'La La Land', Decimal('4.25'))
(22, 'Sing 2', Decimal('4.25'))
(4, 'The Suicide Squad', Decimal('4.25'))
(7, 'On Your Wedding Day', Decimal('4.17'))
(1, 'John Wick: Chapter 4', Decimal('2.86'))
(11, 'West Side Story', None)
(24, 'Minions', None)
(30, 'Procession', None)
(6, 'Shang-Chi and the Legend of the Ten Rings', None)
(18, 'Last Night in Soho', None)
(12, 'A Journal for Jordan', None)
```

```
>>> Rank monthly

(30, 'Procession', Decimal('4.00'))
(11, 'West Side Story', Decimal('3.67'))
(10, 'La La Land', Decimal('3.58'))
(7, 'On Your Wedding Day', Decimal('3.44'))
(8, 'Decision to Leave', Decimal('3.43'))
(12, 'A Journal for Jordan', Decimal('3.43'))
(9, 'Begin Again', Decimal('3.22'))
(29, 'The Sparks Brothers', Decimal('3.13'))
(19, 'Encanto', Decimal('3.00'))
(23, 'Madagascar', Decimal('2.88'))
(1, 'John Wick: Chapter 4', Decimal('2.86'))
(21, 'Zootopia', Decimal('2.85'))
(22, 'Sing 2', Decimal('2.83'))
(15, 'Malignant', Decimal('2.67'))
(20, 'Luca', Decimal('2.60'))
(4, 'The Suicide Squad', Decimal('2.58'))
(17, 'The Wailing', Decimal('2.50'))
(6, 'Shang-Chi and the Legend of the Ten Rings', Decimal('2.43')
(13, 'A Quiet Place Part II', Decimal('2.38'))
(2, 'No Time to Die', Decimal('2.38'))
(28, 'Val', Decimal('2.31'))
(25, 'The Rescue', Decimal('2.30'))
(24, 'Minions', Decimal('2.25'))
(27, 'Flee', Decimal('2.19'))
(18, 'Last Night in Soho', Decimal('2.06'))
(16, 'It', Decimal('2.00'))
(14, 'The Conjuring: The Devil Made Me Do It', Decimal('2.00'))
(3, 'Black Widow', Decimal('2.00'))
(26, 'Summer of Soul', Decimal('2.00'))
(5, 'F9', Decimal('2.00'))
```

갱신

Trigger

```
Possible commands:
-----------------------------------------------
★ Fetch [ user / movie / rating ]
★ Show [ users / movies / ratings ]
★ Display [ user history / movie ratings ]
-----------------------------------------------
★ Insert [ user / movie / rating ]
★ Update [ user / movie / rating ]
★ Delete [ user / movie / rating ]
-----------------------------------------------
★ Rank [ weekly / monthly ]
-----------------------------------------------
★ Get [ cu / cold-start users ]
★ Rec [ cb / content-based ]
★ Rec [ cf / collaborative filtering ]
-----------------------------------------------
★ Quit

>>>
```

5. Main Scenario 1

Movie의 Rating에 기반한 Weekly 혹은 Monthly Ranking의 정보를 요청하는 경우

```
>>> Rank weekly

(29, 'The Sparks Brothers', None)
(4, 'The Suicide Squad', None)
(10, 'La La Land', None)
(6, 'Shang-Chi and the Legend of the Ten Rings', None)
(14, 'The Conjuring: The Devil Made Me Do It', None)
(22, 'Sing 2', None)
(13, 'A Quiet Place Part II', None)
(2, 'No Time to Die', None)
(16, 'It', None)
(11, 'West Side Story', None)
(9, 'Begin Again', None)
(7, 'On Your Wedding Day', None)
(15, 'Malignant', None)
(26, 'Summer of Soul', None)
(12, 'A Journal for Jordan', None)
(24, 'Minions', None)
(19, 'Encanto', None)
(25, 'The Rescue', None)
(30, 'Procession', None)
(21, 'Zootopia', None)
(3, 'Black Widow', None)
(17, 'The Wailing', None)
(28, 'Val', None)
(20, 'Luca', None)
(1, 'John Wick: Chapter 4', None)
(5, 'F9', None)
(18, 'Last Night in Soho', None)
(27, 'Flee', None)
(23, 'Madagascar', None)
(8, 'Decision to Leave', None)
```

```
>>> Rank weekly

(5, 'F9', Decimal('5.00'))
(17, 'The Wailing', Decimal('5.00'))
(19, 'Encanto', Decimal('4.83'))
(27, 'Flee', Decimal('4.75'))
(29, 'The Sparks Brothers', Decimal('4.75'))
(21, 'Zootopia', Decimal('4.75'))
(13, 'A Quiet Place Part II', Decimal('4.75'))
(15, 'Malignant', Decimal('4.75'))
(25, 'The Rescue', Decimal('4.67'))
(8, 'Decision to Leave', Decimal('4.50'))
(26, 'Summer of Soul', Decimal('4.50'))
(3, 'Black Widow', Decimal('4.50'))
(23, 'Madagascar', Decimal('4.50'))
(28, 'Val', Decimal('4.25'))
(16, 'It', Decimal('4.25'))
(2, 'No Time to Die', Decimal('4.25'))
(9, 'Begin Again', Decimal('4.25'))
(14, 'The Conjuring: The Devil Made Me Do It', Decimal('4.25'))
(20, 'Luca', Decimal('4.25'))
(10, 'La La Land', Decimal('4.25'))
(22, 'Sing 2', Decimal('4.25'))
(4, 'The Suicide Squad', Decimal('4.25'))
(7, 'On Your Wedding Day', Decimal('4.17'))
(1, 'John Wick: Chapter 4', Decimal('2.86'))
(11, 'West Side Story', None)
(24, 'Minions', None)
(30, 'Procession', None)
(6, 'Shang-Chi and the Legend of the Ten Rings', None)
(18, 'Last Night in Soho', None)
(12, 'A Journal for Jordan', None)
```

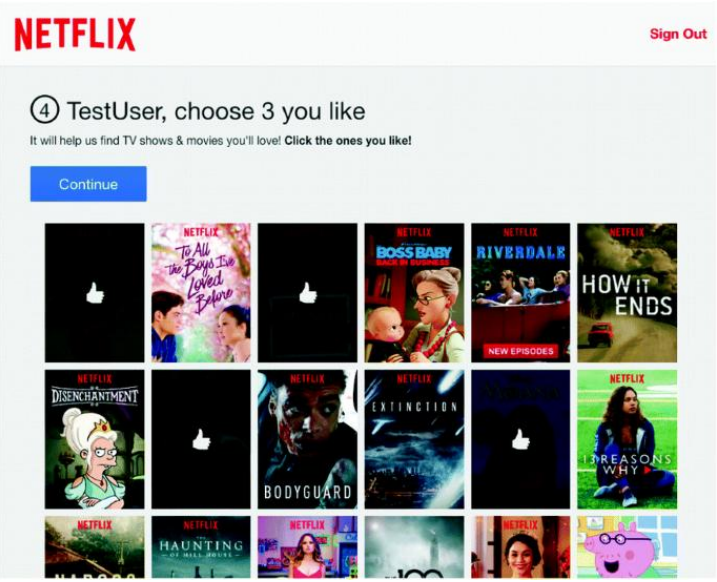
5. Main Scenario 1

Movie의 Rating에 기반한 Weekly 혹은 Monthly Ranking의 정보를 요청하는 경우

```python
def create_triggers(cur, type):                                                 # Keyboard Input을 통해 받은 주간 랭킹 요청을 수행하기 위한 트리거를 갱신하는 함수

    if(type == "month"):
        create_function_monthly = """
        CREATE OR REPLACE FUNCTION refresh_monthly_ranking() RETURNS TRIGGER AS $$          # create_function_monthly 정의
        BEGIN
            EXECUTE '                                                                       # 월간 순위 뷰를 갱신하는 SQL 쿼리문 생성
                CREATE OR REPLACE VIEW monthly_ranking AS                                   # 월간 순위 뷰 생성 혹은 갱신
                SELECT m.mid, m.title, ROUND(AVG(r.rating::numeric), 2) AS avg_rating       # movie ID, movie title, 평균 평점 정보를 이용
                FROM movies m
                LEFT JOIN ratings r ON m.mid = r.mid AND r.time >= (CURRENT_DATE - 30)      # 테이블에 저장된 평점 날짜를 이용하여 최근 30일 평점만 고려
                GROUP BY m.mid, m.title
                ORDER BY avg_rating DESC NULLS LAST;                                        # 평균 평점은 내림차순 정렬로 나타내고 평점 없는 영화는 마지막으로 배치
            ';
            RETURN NULL;                                                                    # 트리거 함수는 결과를 반환하지 않음
        END;
        $$ LANGUAGE plpgsql;                                                                # PostgreSQL을 이용
        """
        cur.execute(create_function_monthly)                                                # SQL 쿼리 실행

        cur.execute("DROP TRIGGER IF EXISTS refresh_monthly_ranking_trigger_movies ON movies;")     # 기존 트리거 삭제
        create_trigger_movies_monthly = """
            CREATE TRIGGER refresh_monthly_ranking_trigger_movies                           # 트리거 refresh_monthly_ranking_trigger_movies 생성
            AFTER INSERT OR UPDATE OR DELETE ON movies                                      # INSERT, UPDATE, DELETE 발생 시 트리거 실행
            FOR EACH STATEMENT                                                              # 각 행 실행 후에 트리거 실행
            EXECUTE FUNCTION refresh_monthly_ranking();                                     # refresh_monthly_ranking 함수 호출
        """
        cur.execute(create_trigger_movies_monthly)                                          # SQL 쿼리 실행

        cur.execute("DROP TRIGGER IF EXISTS refresh_monthly_ranking_trigger_ratings ON ratings;")     # 기존 트리거 삭제
        create_trigger_ratings_monthly = """
            CREATE TRIGGER refresh_monthly_ranking_trigger_ratings                          # 트리거 refresh_monthly_ranking_trigger_ratings 생성
            AFTER INSERT OR UPDATE OR DELETE ON ratings                                     # INSERT, UPDATE, DELETE 발생 시 트리거 실행
            FOR EACH STATEMENT                                                              # 각 행 실행 후에 트리거 실행
            EXECUTE FUNCTION refresh_monthly_ranking();                                     # refresh_monthly_ranking 함수 호출
        """
        cur.execute(create_trigger_ratings_monthly)                                         # SQL 쿼리 실행

def show_monthly_ranking():                                                                 # Keyboard Input을 통해 받은 주간 랭킹 요청을 나타내는 함수
    conn = connect_to_db()                                                                  # PostgreSQL에 연결
    cur = conn.cursor()                                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성
    cur.execute("""
        CREATE OR REPLACE VIEW monthly_ranking AS
        SELECT m.mid, m.title, ROUND(AVG(r.rating)::numeric, 2) AS avg_rating
        FROM movies m
        LEFT JOIN ratings r ON m.mid = r.mid AND r.time >= (CURRENT_DATE - 30)
        GROUP BY m.mid, m.title
        ORDER BY avg_rating DESC NULLS LAST;
    """)
    create_triggers(cur, "month")                                                           # 트리거 갱신을 위한 함수인 create_function_monthly의 조건과 동일
    cur.execute("SELECT * FROM monthly_ranking;")                                           # 트리거 갱신을 위한 함수 호출
    results = cur.fetchall()                                                                # SQL 쿼리 실행
    disconnect(cur, conn)                                                                   # 조회된 모든 행 가져오기
    return results                                                                          # PostgreSQL에 연결 종료
                                                                                            # 조회된 정보 리턴
```

5. Main Scenario 2

Cold-start Users를 위한 Content-based Recommendation을 요청 받은 경우



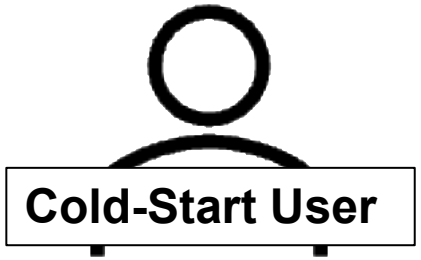

```
>>> Get cold-start users

(11, 'Rucy', 'F', 33, 1)
(15, 'Ana', 'F', 20, 1)
(14, 'Tomas', 'M', 40, 1)
(13, 'Elly', 'F', 29, 1)
(12, 'Luca', 'M', 26, 1)
```

```
>>> Insert user

name:  PSG
gender [F/M/U]: M
age: 31

Inserted user: (16, 'PSG', 'M', 31)

>>> Insert rating

user id: 16
movie id: 1
rating: 5
time: 2024-06-12

Inserted rating: (16, 1, 5.0, datetime.date(2024, 6, 12))
```

```
>>> Get cold-start users

(11, 'Rucy', 'F', 33, 1)
(15, 'Ana', 'F', 20, 1)
(14, 'Tomas', 'M', 40, 1)
(13, 'Elly', 'F', 29, 1)
(16, 'PSG', 'M', 31, 1)
(12, 'Luca', 'M', 26, 1)
```

5. Main Scenario 2

Cold-start Users를 위한 Content-based Recommendation을 요청 받은 경우



Cold-Start User

User relationship

User가 평가한 컨텐츠가 매우 적음

Content-based Recommendation

```python
cur.execute(query3, (uid,))
array3 = np.array(cur.fetchall()).squeeze(axis=1)
array3 = np.expand_dims(array3, axis=1)
array3 = np.tile(array3, (1, num_unrated))

k = 3
array = np.multiply(np.dot(array1, array2.T), array3)
array = np.sum(array, axis=0)
index_order = np.argsort(array)[-k:][::-1].tolist()
index_order = [x + 1 for x in index_order]
 # SQL 쿼리 실행: User가 Rating한 영화들의 평점을 추출
 # User가 Rating한 영화들의 Rating을 numpy array로 저장
 # Rating array을 2차원으로 변환
 # array3: Rating 배열을 User가 평가하지 않은 영화 개수만큼 반복하여 2차원 array 생성

 # 추천할 영화 개수
 # rate한 영화들과 (array1) rate하지 않은 영화들의 (array2) embedding들을 dot product한 후 array3을 활용한 rate만큼 가중치를 곱함
 # weighted sum하여 최종 array 생성
 # index_order: Cosine Similarity을 기준으로 내림차순 정렬된 index_order 생성
 # 인덱스 1부터 시작하도록 조정
```

```python
def recommend_contents_based(uid):                                  # Keyboard Input을 통해 받은 contents-based 추천 요청을 수행하기 위한 함수
    conn = connect_to_db()                                                  # PostgreSQL에 연결
    cur = conn.cursor()                                                     # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성

    query1 = """
        SELECT m.embedding
        FROM ratings r
        JOIN users u ON r.uid = u.uid
        JOIN movies m ON r.mid = m.mid
        WHERE u.uid = %s
        ORDER BY r.mid """
    cur.execute(query1, (uid,))                                             # SQL 쿼리 실행: User가 Rating한 영화들의 embedding 벡터 추출
    array1 = np.array(cur.fetchall()).squeeze(axis=1)                       # array1: User가 Rating한 영화들의 embedding 벡터들을 numpy 배열로 저장

    query2 = """
        WITH rated_movies AS (
            SELECT r.mid
            FROM ratings r
            JOIN users u ON r.uid = u.uid
            WHERE u.uid = %s
            ORDER BY r.mid
        )
        SELECT m.embedding
        FROM movies m
        WHERE m.mid NOT IN (SELECT mid FROM rated_movies)
        ORDER BY m.mid
        """
    cur.execute(query2, (uid,))                                             # SQL 쿼리 실행: User가 Rating하지 않은 영화들의 embedding 벡터 추출
    array2 = np.array(cur.fetchall()).squeeze(axis=1)                       # array2: User가 Rating하지 않은 영화들의 embedding 벡터들을 numpy 배열로 저장
    num_unrated = array2.shape[0]                                           # User가 Rating하지 않은 영화들의 개수 저장

    query3 = """
        SELECT r.rating
        FROM ratings r
        JOIN users u ON r.uid = u.uid
        WHERE u.uid = %s
        ORDER BY r.mid """
    cur.execute(query3, (uid,))                                             # SQL 쿼리 실행: User가 Rating한 영화들의 평점을 추출
    array3 = np.array(cur.fetchall()).squeeze(axis=1)                       # User가 Rating한 영화들의 평점을 numpy 배열로 저장
    array3 = np.expand_dims(array3, axis=1)                                 # 평점 배열을 2차원으로 변환
    array3 = np.tile(array3, (1, num_unrated))                              # array3: 평점 배열을 User가 평가하지 않은 영화 개수만큼 반복하여 2차원 배열 생성

    k = 3                                                                   # 추천할 영화 개수
    array = np.multiply(np.dot(array1, array2.T), array3)                   # rate한 영화들과 (array1) rate하지 않은 영화들의 (array2) embedding들을 dot product한 후 rate만큼 가중치를 곱함
    array = np.sum(array, axis=0)                                           # weighted sum하여 최종 array 생성
    index_order = np.argsort(array)[-k:][::-1].tolist()                     # index_order: 유사도 합을 기준으로 내림차순 정렬된 index_order 생성
    index_order = [x + 1 for x in index_order]                              # 인덱스 1부터 시작하도록 조정
    query = f"""
        WITH rated_movies AS (
            SELECT r.mid
            FROM ratings r
            JOIN users u ON r.uid = u.uid
            WHERE u.uid = {uid}
            ORDER BY r.mid
        ),
        unrated_movies AS (
            SELECT m.mid, m.title, m.director, m.nationality, m.release_year, m.profit, m.summary, ROW_NUMBER() OVER (ORDER BY m.mid) AS row_num
            FROM movies m
            WHERE m.mid NOT IN (SELECT mid FROM rated_movies)
            ORDER BY m.mid
        ),
        selected_movies AS (
            SELECT *
            FROM unrated_movies
            WHERE row_num IN {tuple(index_order)}
        )
        SELECT mid, title, director, nationality, release_year, profit, summary
        FROM selected_movies
        ORDER BY CASE row_num
            {''.join([f'WHEN {i} THEN {index_order.index(i)} ' for i in index_order])}
            ELSE row_num
        END; """

    cur.execute(query)                                                      # SQL 쿼리 실행: 앞서 생성한 index_order를 통해 contents-based 추천영화 K개를 추출
    result = cur.fetchall()                                                 # 조회된 모든 행 가져오기
    disconnect(cur, conn)                                                   # PostgreSQL에 연결 종료
    return result                                                           # 조회된 정보 리턴
```

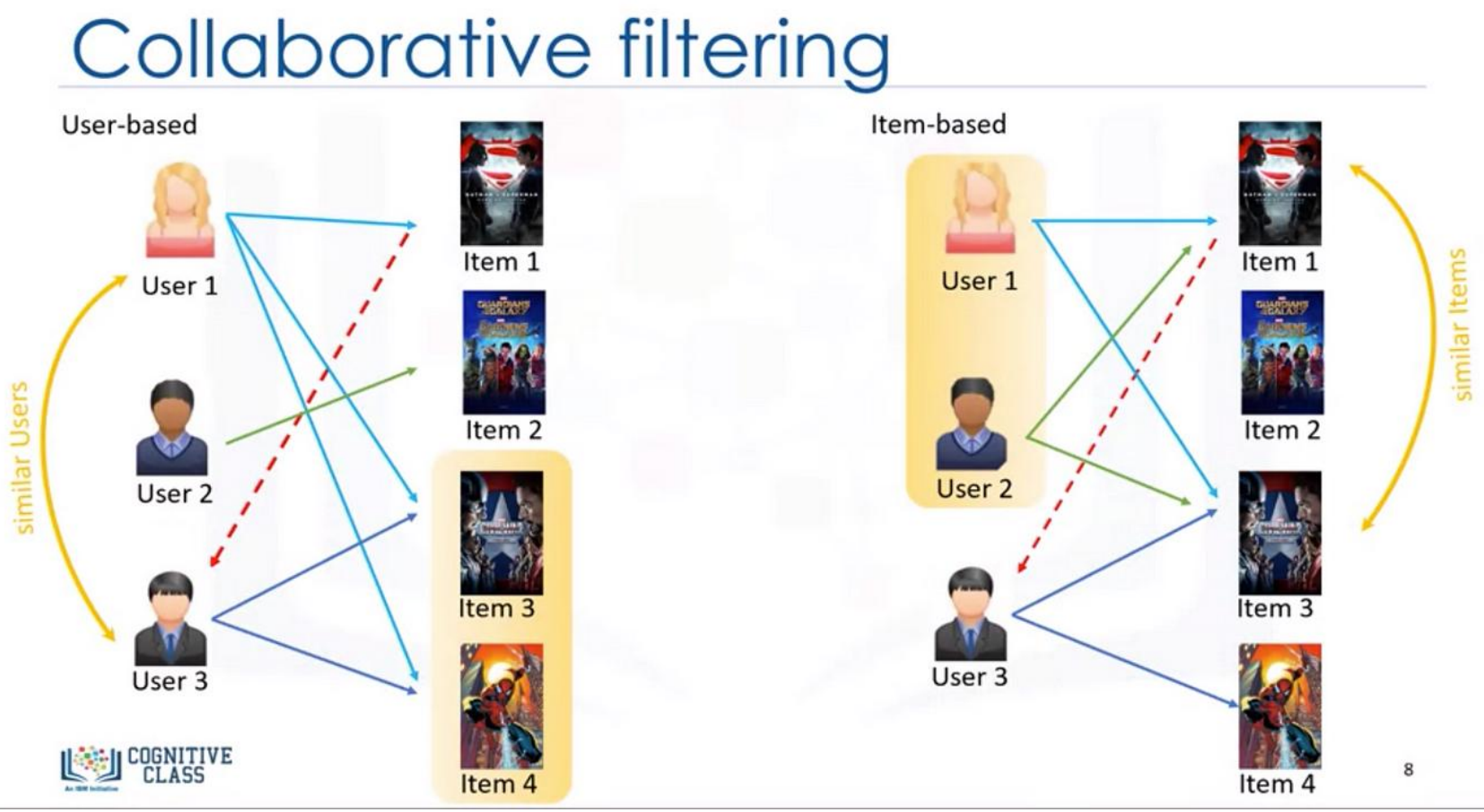
5. Main Scenario 2

Cold-start Users를 위한 Content-based Recommendation을 요청 받은 경우

```
>>> Rec content-based

user id: 16

User information: (16, 'PSG', 'M', 31)

Activity history:
(1, 'John Wick: Chapter 4', 5.0, datetime.date(2024, 6, 12))

Recommended Movies:
(5, 'F9', 'Justin Lin', 'USA', 2021, 726000000, 'Dom Toretto and his crew face new threats from the past. High-speed car chases and action scenes.')
(2, 'No Time to Die', 'Cary Joji Fukunaga', 'USA', 2021, 774000000, 'James Bond comes out of retirement to face a new villain. Thrilling espionage action unfolds.')
(4, 'The Suicide Squad', 'James Gunn', 'USA', 2021, 167000000, 'A new team of anti-heroes embark on a dangerous mission. Dark humor and action abound.')
```

5. Main Scenario 3

Collaborative Filtering Recommendation을 요청 받은 경우



How to estimate similarity?

https://medium.com/@toprak.mhmt/collaborative-filtering-3ceb89080ade

5. Main Scenario 3

Collaborative Filtering Recommendation을 요청 받은 경우

```python
def recommend_collaborative_filtering(uid):                                    # Keyboard Input을 통해 받은 collaborative filtering 추천 요청을 수행하기 위한 함수
    conn = connect_to_db()                                                         # PostgreSQL에 연결
    cur = conn.cursor()                                                            # SQL 쿼리 실행 및 결과 처리를 위한 커서 생성

    target_index = int(uid) - 1                                                    # 추천 대상 User의 인덱스 계산

    cur.execute("SELECT COUNT(DISTINCT uid) FROM users")                           # SQL 쿼리 실행: User 수 계산
    num_users = cur.fetchone()[0]
    cur.execute("SELECT COUNT(DISTINCT mid) FROM movies")                          # SQL 쿼리 실행: Movie 수 계산
    num_movies = cur.fetchone()[0]
    rating_matrix = np.zeros((num_users, num_movies))                              # Rating matrix 생성
    cur.execute("SELECT uid, mid, rating FROM ratings")                            # SQL 쿼리 실행: 모든 Rating 데이터 가져오기
    ratings = cur.fetchall()
    for uid, mid, rating in ratings: rating_matrix[uid-1][mid-1] = rating          # Rating matrix에 Rating 데이터 매칭시키기

    similarity_matrix = cosine_similarity(rating_matrix)                           # Rating matrix의 cosine_similarity를 도출하여 similarity_matrix 생성
    user_similarities = similarity_matrix[target_index]                            # 해당 User와 다른 User들의 similarity를 similarity_matrix를 이용하여 구하기
    most_similar_user_index = np.argsort(user_similarities)[-2]                    # 해당 User와 가장 유사한 User의 인덱스 추출
    target_user_ratings = rating_matrix[target_index]                              # 해당 User의 Rating 정보
    similar_user_ratings = rating_matrix[most_similar_user_index]                  # 가장 유사한 User의 Rating 정보

    high_rated_movies = np.where(similar_user_ratings >= 4.5)[0]                   # 가장 유사한 User가 4.5 이상으로 Rating한 Movie의 인덱스
    unwatched_movies = np.where(target_user_ratings == 0)[0]                       # 해당 User가 아직 보지 않은 영화 인덱스
    recommended_movies = np.intersect1d(high_rated_movies, unwatched_movies)       # 교집합 추출

    if len(recommended_movies) > 0:                                                # 추천 Movie가 있을 경우
        recommended_movies_list = recommended_movies.tolist()                          # numpy array를 리스트로 변환
        recommended_movies_list = [int(mid) + 1 for mid in recommended_movies_list]    # 각 요소를 int로 변환하여 mid 생성
        format_strings = ','.join(['%s'] * len(recommended_movies_list))
        query = f"SELECT mid, title, director, nationality, release_year, profit, summary FROM movies WHERE mid IN ({format_strings})"
        cur.execute(query, tuple(recommended_movies_list))                             # SQL 쿼리 실행: 추천 영화 정보 조회
        movies = cur.fetchall()
    else:
        movies = []                                                                    # 추천 영화가 없을 경우 빈 리스트 반환

    disconnect(cur, conn)                                                          # PostgreSQL에 연결 종료
    return movies                                                                  # 조회된 정보 리턴
```

User 수와 Movie 수를 고려한 Rating Matrix를 생성시켜 Rating 데이터와 매칭
cosine similarity를 계산하여 만들어진 similarity matrix 생성
해당 User와 다른 User간의 similarity를 도출

5. Main Scenario 3

Collaborative Filtering Recommendation을 요청 받은 경우

```
>>> Insert rating

user id: 16
movie id: 9
rating: 4.5
time: 2024-06-12

Inserted rating: (16, 9, 4.5, datetime.date(2024, 6, 12))
>>> Insert rating

user id: 16
movie id: 10
rating: 5
time: 2024-06-12

Inserted rating: (16, 10, 5.0, datetime.date(2024, 6, 12))
>>> Insert rating

user id: 16
movie id: 7
rating: 4
time: 2024-06-12

Inserted rating: (16, 7, 4.0, datetime.date(2024, 6, 12))
```

```
>>> Rec collaborative filtering

user id: 16

User information: (16, 'PSG', 'M', 31)

Preferred Movies:
(1, 'John Wick: Chapter 4', 5.0, datetime.date(2024, 6, 12))
(9, 'Begin Again', 4.5, datetime.date(2024, 6, 12))
(10, 'La La Land', 5.0, datetime.date(2024, 6, 12))
(7, 'On Your Wedding Day', 4.0, datetime.date(2024, 6, 12))

Recommended Movies:
(8, 'Decision to Leave', 'Park Chan-wook', 'South Korea', 2022, 45000000, "A detective inve
stigating a man's death in the mountains ends up meeting and developing feelings for the de
ad man's mysterious wife in the course of his dogged sleuthing. A romance film.")
(12, 'A Journal for Jordan', 'Denzel Washington', 'USA', 2021, 6500000, 'A soldier leaves a
 journal for his son. A love story told through letters.')
```

6. Achievements through the project

2024-23352 한정관

필요 과제를 나누어 서로 분업해서 프로젝트를 진행하며 협업의 필요성과 효율성을 이해하게 됨
시중에서 얻거나 부여할 수 있는 데이터의 특성이나 양이 많다고 해서 주어진 프로젝트에
모두 쓸모있는 특징을 지닌 것은 아니라는 것을 알게 됨

2024-28634 김주영

추천시스템(그래프)을 공부하는 학생으로서,
실제 추천 환경이 어떻게 돌아가는지 살펴보고 직접 구현까지 해볼 수 있었던 기회인 만큼
앞으로의 연구에 밑거름이 되어줄 경험을 한지라 기뻤다.

2024-25545 이서혁

백엔드 애플리케이션을 개발해 본 경험이 전혀 없었는데,
이번 과제를 통해 조원 분들과 실제로 구현해보는 시간을 가지면서 백엔드 애플리케이션의
전반적인 구동 원리 및 각 구성 요소의 역할을 이해하게 되었습니다.

2022-26826 박승과

바이오인포 분야를 연구하는 사람으로서 다시 한번 협업의 중요성을 많이 느꼈고
평상시 연구에는 경험하지 못할 백엔드 어플리케이션 구축을 과제로나마 흐름을 알게 되어 좋았다.

백엔드 어플리케이션 구축 경험

협업의 필요성과 효율성